FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

4

IMPLEMENTING KNOWLEDGE REPRESENTATION, RULE-BASED REASONING (RBR), AND CASE-BASED REASONING (CBR)

Student Name	SANGKULA, Chedrick Khylle P.
Section:	[Section]
Professor:	[Professor]
Date:	November 3, 2026

I. BACKGROUND INFORMATION

Knowledge Representation (KR) is the method of organizing facts, rules, and relationships so an AI system can understand and reason about the world.

Reasoning (R) allows the system to infer, make decisions, and solve problems based on that knowledge. Two major reasoning approaches from the module:

1. Rule-Based Reasoning (RBR)

Uses IF–THEN rules, a Knowledge Base, Working Memory, and an Inference Engine.

2. Case-Based Reasoning (CBR)

Solves new problems using past cases using the 4R Cycle:

Retrieve → Reuse → Revise → Retain

This laboratory lets you build small working implementations of both reasoning methods.

II. TECHNICAL ASSESSMENT

IMPLEMENTING KNOWLEDGE REPRESENTATION, RULE-BASED REASONING (RBR), AND CASE-BASED REASONING (CBR)

Objective

At the end of this laboratory activity, students are expected to:

1. Construct a simple knowledge base using Python data structures.
2. Implement a basic Rule-Based Reasoning (RBR) system using IF–THEN logic.
3. Build a simple Case-Based Reasoning (CBR) system using similarity matching.

4. Compare how RBR and CBR behave when solving the same type of problem.
5. Document observations and insights on KR, RBR, and CBR.

Materials Needed

1. Hardware
   - A laptop or desktop computer
   - Stable internet connection (for research, documentation, submission)
2. Software

A programming environment:

- Python (recommended: IDLE, VS Code, Jupyter Notebook, PyCharm)
- Any text editor (Notepad++, Sublime, VS Code) if needed

3. File Formats & Tools

Word processor for documentation:

- MS Word, Google Docs, or LibreOffice
- Screenshot tool (e.g., Snipping Tool, Lightshot, Print Screen)

General Instructions

1. This **Technical Assessment (TA3)** must be completed by Pair.
2. Include clear comments in your code to describe its logic.
3. Each program should display the correct input, process, and output.
4. Ensure that your script displays clear and readable outputs.
5. Screenshot of program output for RBR
6. Screenshot of program output for CBR
7. Reflection answers
8. Short description of your knowledge base and use case
9. Compile all Python codes and corresponding outputs into one PDF file.
10. Save it as **TA4_lastname1.pdf**

    (Example: TA2_Patilano.pdf)
11. Submit the **python file** and **PDF** file.

LABORATORY TASKS

PART 1 — Knowledge Representation (KR) Setup

Task 1A — Create a Knowledge Base

Create a Python dictionary storing at least:

- 5 facts about an object (e.g., weather, appliances, students, medical symptoms)
- 3 categories (e.g., temperature, status, presence, actions)

Example structure to follow:

```
facts = {
    "temperature": 32,
    "is_raining": False,
    "humidity": 85,
}
```

PART 2 — Implement Rule-Based Reasoning (RBR)

Task 2A — Create at least 3 IF–THEN rules

Write Python functions representing rules.

Example:

```
def rule_hot(facts):
    if facts["temperature"] > 30:
        return "Turn on the aircon"
```

Task 2B — Implement an Inference Engine

Create a loop that:

1. Reads the knowledge base
2. Checks each rule
3. Returns all actions inferred

Task 2C — Display Output

Example [Program should print:]

```
Rule-Based Reasoning Actions:
- Turn on the aircon
- Bring umbrella
```

PART 3 — Implement Case-Based Reasoning (CBR)

Task 3A — Build a Case Base

Create at least **3 past cases**, each containing:

- A set of problem features

- A matching solution

Example:

```
case_base = [
    {"problem": {"won't turn on": True, "noise": False},
     "solution": "Check power cable."}
]
```

Task 3C — Reuse Step

Return the solution of the most similar case.

Task 3D — Revise Step

Allow student to adjust solution manually or return unchanged.

Task 3E — Retain Step

Add the new case to the case base.

PART 4 — Comparative Reflection

Write a short paragraph answering:

1. Which system (RBR or CBR) was easier to implement? Why?
2. Which is more flexible in solving unstructured problems?
3. Which is better for situations with continuously changing data?
4. Examples of real-world applications for both.

Your Code - Laboratory Tasks ⇩

Short Description of the Knowledge Base and Use Case

Use case: a Laptop Troubleshooting Assistant that diagnoses a slow, overheating laptop. The same problem is solved twice, once with expert IF–THEN rules (RBR) and once by matching it to past repair cases (CBR), so the two reasoning methods can be compared side by side.

The knowledge base is a Python dictionary with 11 facts grouped into three categories. Status holds true/false states (is_plugged_in, is_charging, wifi_connected), measurements holds numeric readings (battery_percent, cpu_temperature_c, free_storage_gb, ram_usage_percent), and symptoms holds what the user observes

(fan_noise_loud, runs_slow, screen_flicker, wont_turn_on). A helper function get_fact() looks up any fact by name so the rules do not need to know which category it is in.

Category	Facts	Current values
status	is_plugged_in, is_charging, wifi_connected	False, False, False
measurements	battery_percent, cpu_temperature_c, free_storage_gb, ram_usage_percent	12, 92, 4, 91
symptoms	fan_noise_loud, runs_slow, screen_flicker, wont_turn_on	True, True, False, False

How the RBR and CBR Parts Work

For RBR, seven rules are written as Python functions, such as IF cpu_temperature_c > 85 AND fan_noise_loud THEN clean the vents and fan. The inference engine loops through every rule, checks it against the knowledge base, and collects the action of each rule that fires. With the current facts, 5 of the 7 rules fire.

For CBR, the case base starts with four past repairs, each with six true/false problem features and the solution that worked. The knowledge base is converted into the same six features, and similarity is the number of matching features divided by six. The most similar case is retrieved (Case 1 at 83%) and its solution is reused. In the revise step the user can type a modified solution or press Enter to keep it, and the retain step adds the new case so the case base grows from four to five cases.

Python Code (kr_rbr_cbr_laptop.py)

```
"""
TA4 - Implementing Knowledge Representation (KR),
      Rule-Based Reasoning (RBR), and Case-Based Reasoning (CBR)

Use case: LAPTOP TROUBLESHOOTING ASSISTANT
The same laptop problem is solved in two ways:
  - RBR: expert IF-THEN rules fire on the current facts
  - CBR: the most similar past repair case is retrieved, reused,
         revised (optional), and retained

Run:
    python kr_rbr_cbr_laptop.py
"""

# ======================================================================
# PART 1 - KNOWLEDGE REPRESENTATION (KR)
# Task 1A: Knowledge base stored as a Python dictionary.
# Facts are grouped into 3 categories: status, measurements, symptoms.
# ======================================================================
knowledge_base = {
    "status": {                     # Category 1: current state of the laptop
        "is_plugged_in": False,
        "is_charging": False,
        "wifi_connected": False,
    },
    "measurements": {               # Category 2: numeric readings
        "battery_percent": 12,
        "cpu_temperature_c": 92,
        "free_storage_gb": 4,
        "ram_usage_percent": 91,
    },
    "symptoms": {                   # Category 3: what the user observes
```

```
        "fan_noise_loud": True,
        "runs_slow": True,
        "screen_flicker": False,
        "wont_turn_on": False,
    },
}


def get_fact(kb, name):
    """Look up a fact by name, no matter which category it is in."""
    for category in kb.values():
        if name in category:
            return category[name]
    raise KeyError(f"Unknown fact: {name}")


def show_knowledge_base(kb):
    print("KNOWLEDGE BASE (facts about the laptop)")
    print("-" * 60)
    for category, facts in kb.items():
        print(f"[{category.upper()}]")
        for name, value in facts.items():
            print(f"   {name:<22}: {value}")
    print()


# ======================================================================
# PART 2 - RULE-BASED REASONING (RBR)
# Task 2A: IF-THEN rules written as Python functions.
# Each rule returns an action if its condition is true, else None.
# ======================================================================
def rule_overheating(kb):
    # IF CPU temperature > 85 C AND fan is loud THEN clean vents/fans
    if get_fact(kb, "cpu_temperature_c") > 85 and get_fact(kb, "fan_noise_loud"):
        return "Overheating: clean the vents/fan and use a cooling pad"


def rule_low_battery(kb):
    # IF battery < 20% AND not plugged in THEN plug in the charger
    if get_fact(kb, "battery_percent") < 20 and not get_fact(kb, "is_plugged_in"):
        return "Low battery: plug in the charger now"


def rule_charger_fault(kb):
    # IF plugged in BUT not charging THEN check charger/adapter
    if get_fact(kb, "is_plugged_in") and not get_fact(kb, "is_charging"):
        return "Plugged in but not charging: check the adapter and cable"


def rule_low_storage(kb):
    # IF free storage < 10 GB THEN clean up disk
    if get_fact(kb, "free_storage_gb") < 10:
        return "Low storage: delete temp files / uninstall unused apps"


def rule_slow_performance(kb):
    # IF runs slow AND RAM usage > 85% THEN close background apps
    if get_fact(kb, "runs_slow") and get_fact(kb, "ram_usage_percent") > 85:
        return "High RAM usage: close background apps and startup programs"


def rule_no_wifi(kb):
    # IF Wi-Fi is not connected THEN restart the adapter/router
    if not get_fact(kb, "wifi_connected"):
        return "No Wi-Fi: toggle airplane mode or restart the router"
```

```
def rule_no_power(kb):
    # IF laptop won't turn on THEN do a hard reset
    if get_fact(kb, "wont_turn_on"):
        return "Won't turn on: hold the power button for 30s (hard reset)"


RULES = [rule_overheating, rule_low_battery, rule_charger_fault,
         rule_low_storage, rule_slow_performance, rule_no_wifi, rule_no_power]


# Task 2B: Inference engine - reads the KB, checks every rule,
# and collects all actions that were inferred (forward chaining).
def inference_engine(kb, rules):
    actions, fired = [], []
    for rule in rules:
        result = rule(kb)
        if result:                      # condition was true -> rule fired
            fired.append(rule.__name__)
            actions.append(result)
    return actions, fired


# ====================================================================
# PART 3 - CASE-BASED REASONING (CBR)
# Task 3A: Case base of past repairs (problem features + solution).
# ====================================================================
case_base = [
    {"problem": {"wont_turn_on": False, "fan_noise_loud": True,  "runs_slow": True,
                 "overheating": True,  "screen_flicker": False, "battery_low": False},
     "solution": "Cleaned dust from fan and replaced thermal paste."},
    {"problem": {"wont_turn_on": True,  "fan_noise_loud": False, "runs_slow": False,
                 "overheating": False, "screen_flicker": False, "battery_low": True},
     "solution": "Replaced faulty charger; battery recharged normally."},
    {"problem": {"wont_turn_on": False, "fan_noise_loud": False, "runs_slow": True,
                 "overheating": False, "screen_flicker": False, "battery_low": False},
     "solution": "Removed startup programs and upgraded RAM to 16 GB."},
    {"problem": {"wont_turn_on": False, "fan_noise_loud": False, "runs_slow": False,
                 "overheating": False, "screen_flicker": True,  "battery_low": False},
     "solution": "Updated display driver and reseated the screen cable."},
]


def features_from_kb(kb):
    """Convert the knowledge base into CBR problem features."""
    return {
        "wont_turn_on": get_fact(kb, "wont_turn_on"),
        "fan_noise_loud": get_fact(kb, "fan_noise_loud"),
        "runs_slow": get_fact(kb, "runs_slow"),
        "overheating": get_fact(kb, "cpu_temperature_c") > 85,
        "screen_flicker": get_fact(kb, "screen_flicker"),
        "battery_low": get_fact(kb, "battery_percent") < 20,
    }


# Task 3B: Retrieve - similarity = matching features / total features
def similarity(new_problem, old_problem):
    matches = sum(1 for f in new_problem if new_problem[f] == old_problem.get(f))
    return matches / len(new_problem)


def retrieve(new_problem, cases):
    scored = [(similarity(new_problem, c["problem"]), c) for c in cases]
    scored.sort(key=lambda x: x[0], reverse=True)
    return scored


# Task 3C: Reuse - return the solution of the most similar case
```

```
def reuse(best_case):
    return best_case["solution"]


# Task 3D: Revise - user may edit the solution, or press Enter to keep it
def revise(solution):
    print(f"Proposed solution: {solution}")
    edited = input("Revise solution (press Enter to keep unchanged): ").strip()
    if edited:
        print("-> Solution revised by user.")
        return edited
    print("-> Solution kept unchanged.")
    return solution


# Task 3E: Retain - store the new case so the system learns
def retain(cases, problem, solution):
    cases.append({"problem": problem, "solution": solution})
    print(f"-> New case saved! Case base now has {len(cases)} cases.")


def show_case_base(cases):
    for i, c in enumerate(cases, start=1):
        true_feats = [f for f, v in c["problem"].items() if v] or ["(none)"]
        print(f"   Case {i}: {', '.join(true_feats)}")
        print(f"           -> {c['solution']}")


# ======================================================================
# MAIN
# ======================================================================
def main():
    print("=" * 64)
    print("  LAPTOP TROUBLESHOOTING ASSISTANT - KR, RBR and CBR")
    print("=" * 64)

    # ---------- PART 1: KR ----------
    print("\nPART 1 - KNOWLEDGE REPRESENTATION")
    show_knowledge_base(knowledge_base)

    # ---------- PART 2: RBR ----------
    print("=" * 64)
    print("PART 2 - RULE-BASED REASONING")
    print("=" * 64)
    actions, fired = inference_engine(knowledge_base, RULES)
    print(f"Rules checked: {len(RULES)}   Rules fired: {len(fired)}")
    print("Fired rules  : " + ", ".join(fired))
    print("\nRule-Based Reasoning Actions:")
    for a in actions:
        print(f"- {a}")

    # ---------- PART 3: CBR ----------
    print("\n" + "=" * 64)
    print("PART 3 - CASE-BASED REASONING")
    print("=" * 64)
    new_problem = features_from_kb(knowledge_base)
    print("New problem features:")
    for f, v in new_problem.items():
        print(f"   {f:<16}: {v}")

    print("\n[RETRIEVE] Similarity to past cases:")
    ranked = retrieve(new_problem, case_base)
    for score, case in ranked:
        idx = case_base.index(case) + 1
        print(f"   Case {idx}: {score:.0%}  -> {case['solution']}")
    best_score, best_case = ranked[0]
    print(f"Most similar case: Case {case_base.index(best_case) + 1} ({best_score:.0%})")
```

```
    print("\n[REUSE]")
    solution = reuse(best_case)
    print(f"Reused solution from Case {case_base.index(best_case) + 1}: {solution}")

    print("\n[REVISE]")
    final_solution = revise(solution)

    print("\n[RETAIN]")
    retain(case_base, new_problem, final_solution)
    print("\nUpdated case base:")
    show_case_base(case_base)

    print("\nCBR Final Solution:")
    print(f"- {final_solution}")
    print("=" * 64)


if __name__ == "__main__":
    main()
```

Program Output

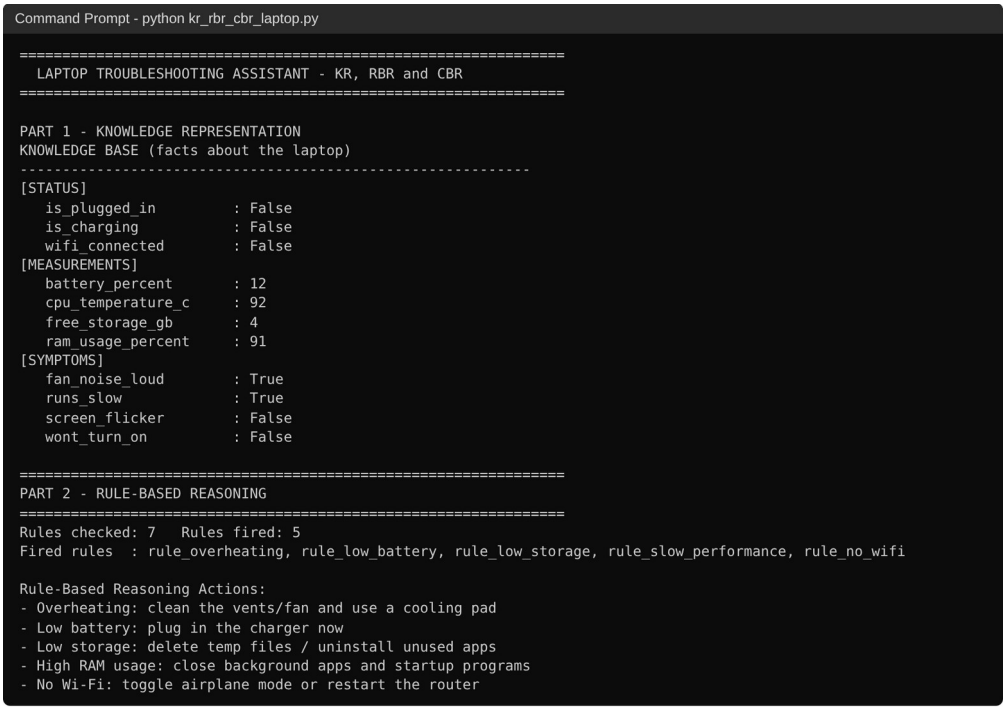


Figure 1. Knowledge base and Rule-Based Reasoning output (5 of 7 rules fired)

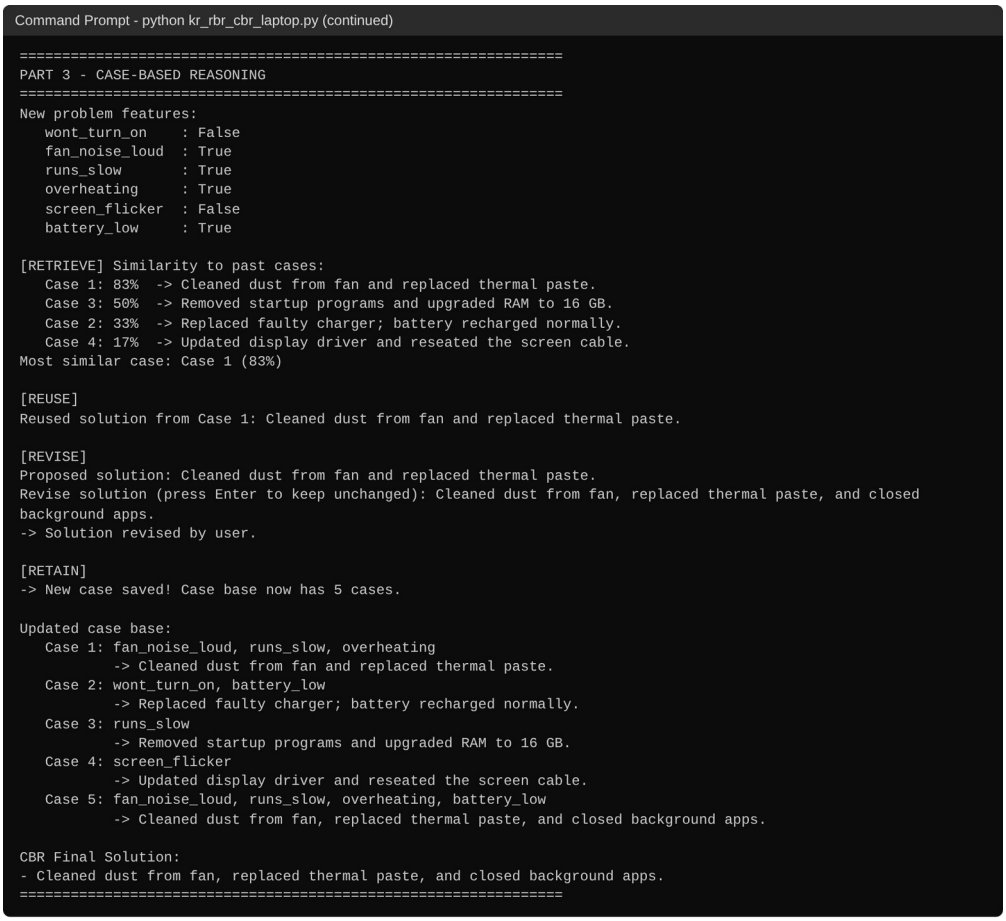


Figure 2. Case-Based Reasoning output — retrieve, reuse, revise (user edit), and retain

Part 4 — Comparative Reflection

RBR was easier to implement because each rule is just an if-statement and the inference engine is a simple loop, while CBR needed a case base, a similarity function, and the four retrieve–reuse–revise–retain steps. However, CBR is more flexible for unstructured problems, since it can still suggest a solution when no rule was written for the exact situation, as long as a similar past case exists. RBR only knows what its rules cover, so a new kind of problem gives no answer until someone writes a new rule.

For continuously changing data, CBR is the better fit because it learns by retaining every solved case, so the system improves without rewriting code, while RBR rules have to be updated by hand whenever conditions change. In the real world, RBR is used in medical expert systems like MYCIN, smart home automation (IF temperature > 30 THEN turn on aircon), and bank fraud alerts, while CBR is used in IT help-desk troubleshooting, legal decision support that retrieves similar court cases, and recommendation systems that suggest items based on similar users.